DEEP LEARNING FUNDAMENTALS • VISUAL TUTORIAL

Loss Functions in Deep Learning

From Zero to Pro

MSE • MAE • Huber • Binary & Categorical Cross-Entropy • Sparse CE
Hinge • KL Divergence • GAN Loss • Focal Loss • Triplet Loss
with diagrams, step-by-step maths, worked examples and Keras code



Rebuilt and expanded from a YouTube lecture script: reorganised, fixed, visualised and extended with advanced topics.

Welcome

This tutorial turns a spoken lecture on loss functions into a structured, visual, step-by-step course. It keeps everything from the lecture and adds what a video cannot show: diagrams, worked numbers, the missing explanations for the losses that were only named (Hinge, KL, GAN, Focal, Triplet), pro-level notes and practice problems.

> **KEY IDEA How to read this tutorial**
> The tutorial moves in five levels. Each topic follows the same recipe: **intuition** (a real-life picture) → **maths** (the formula, explained term by term) → **worked example** (real numbers) → **visual** (a diagram) → **code** (Keras) → **when to use it**. If a formula looks scary, read the intuition first and come back.

Level	Part	You will be able to…
1 · Beginner	Part 1	explain what a loss function is, why training needs it, and the difference between loss and cost
2 · Intermediate	Part 2	use and compare MSE, MAE, Huber for regression problems
3 · Upper-mid	Part 3	use Binary CE, Categorical CE, Sparse CE and Hinge for classification
4 · Advanced	Part 4	understand KL Divergence, GAN loss, Focal loss and Triplet loss
5 · Pro	Part 5	read gradients, avoid numerical traps, choose and write your own loss, ace interviews

Quick glossary (come back here any time)

Word	Plain meaning
Parameters / weights / bias	the numbers inside the model that learning adjusts
Prediction ŷ (“y-hat”)	what the model outputs; y is the true answer
Error	the gap between truth and prediction, $y - \hat{y}$
Epoch	one full pass through the whole training dataset
Gradient / derivative	the slope of the loss: tells which way (and how steeply) the loss changes
Gradient descent	repeatedly stepping the parameters downhill on the loss
Learning rate (η)	the size of each downhill step
Outlier	a data point that breaks the general pattern (e.g. a 50 LPA package among 4–7 LPA ones)
Convex	a bowl-shaped function with one minimum
Logit	a raw score before sigmoid / softmax
One-hot	a label stored as a vector with a single 1, e.g. [0, 1, 0]

Contents

Part 1: Foundations: What a Loss Function Really Is 4

1.1 What is a loss function? 4
1.2 Why is a loss function important? 6
1.3 The role of the loss in deep learning 6
1.4 Loss function vs Cost function 8
1.5 The map of loss functions 9

Part 2: Regression Losses: MSE, MAE and Huber 11

2.0 Setting up a regression network 11
2.1 Mean Squared Error (MSE) 11
2.2 Mean Absolute Error (MAE) 15
2.3 Huber Loss 16
2.4 MSE vs MAE vs Huber at a glance 18

Part 3: Classification Losses: Cross-Entropy and Hinge 20

3.0 Before we start: probabilities and the idea of “surprise” 20
3.1 Binary Cross-Entropy (BCE) 20
3.2 Categorical Cross-Entropy (CCE) 23
3.3 Sparse Categorical Cross-Entropy 25
3.4 Hinge Loss (the SVM loss) 26
3.5 Classification losses at a glance 27

Part 4: Advanced Losses: KL, GAN, Focal and Triplet 28

4.1 KL Divergence (autoencoders and VAEs) 28
4.2 GAN losses: Discriminator loss and Minimax loss 29
4.3 Focal Loss (object detection, imbalanced data) 31
4.4 Triplet Loss (embeddings) 32

Part 5: Pro Level: Gradients, Stability and Choosing Wisely 34

5.1 Why cross-entropy beats MSE for classification 34
5.2 Numerical stability 34
5.3 Loss is not the same as a metric 34
5.4 Regularisation lives inside the loss 35
5.5 Class imbalance 35
5.6 Sanity checks that save hours 35
5.7 Writing your own loss in Keras 35
5.8 Choosing the right loss: the decision map 35
5.9 The master cheat sheet 36
5.10 Interview questions with model answers 37
5.11 Practice problems (answers included) 38
5.12 Did we cover everything in the video? 38

PART 1 • LEVEL 1 · BEGINNER

Foundations: What a Loss Function Really Is

No maths background needed. By the end of this part you will know what a loss function is, why a model cannot learn without one, how it fits inside the training loop, and how loss differs from cost.

1.1 What is a loss function?

In machine learning you build a model that makes **predictions**. The first natural question is: *how good are these predictions?* A **loss function** answers exactly that. It is a method of evaluating how well your algorithm models your dataset.

KEY IDEA Definition in one line

A loss function is a mathematical function that takes the **true value** and the model's **prediction** and returns **one number** that says how wrong the model is. **Big number = bad model. Small number = good model.**

ANALOGY Everyday pictures of a loss

- **Archery:** the distance between the arrow and the bullseye. Closer arrow = smaller loss.
- **Exam:** marks you lost. Fewer lost marks = better student.
- **GPS:** "you are 5 km off the route". The model's job is to bring that number down to zero.

A loss is a function of the model's parameters

A loss function is a mathematical function, just like $f(x) = x^2$. The difference is what you feed into it. In school, x was a free number. In machine learning the input of the loss is the set of **parameters** (weights and biases) of the model. **Change the parameters and the loss changes.**

Example: simple linear regression

Suppose you have some points and you want to fit a straight line through them. The line is:

Equation of the line (m = slope, b = intercept)

$$\hat{y} = m\,x + b$$

Your "model" has just two parameters, m and b. For the loss you use Mean Squared Error (we study it in Part 2). Since every prediction depends on m and b, the loss is a function of them:

Loss as a function of m and b

$$L(m, b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - (m\,x_i + b))^2$$

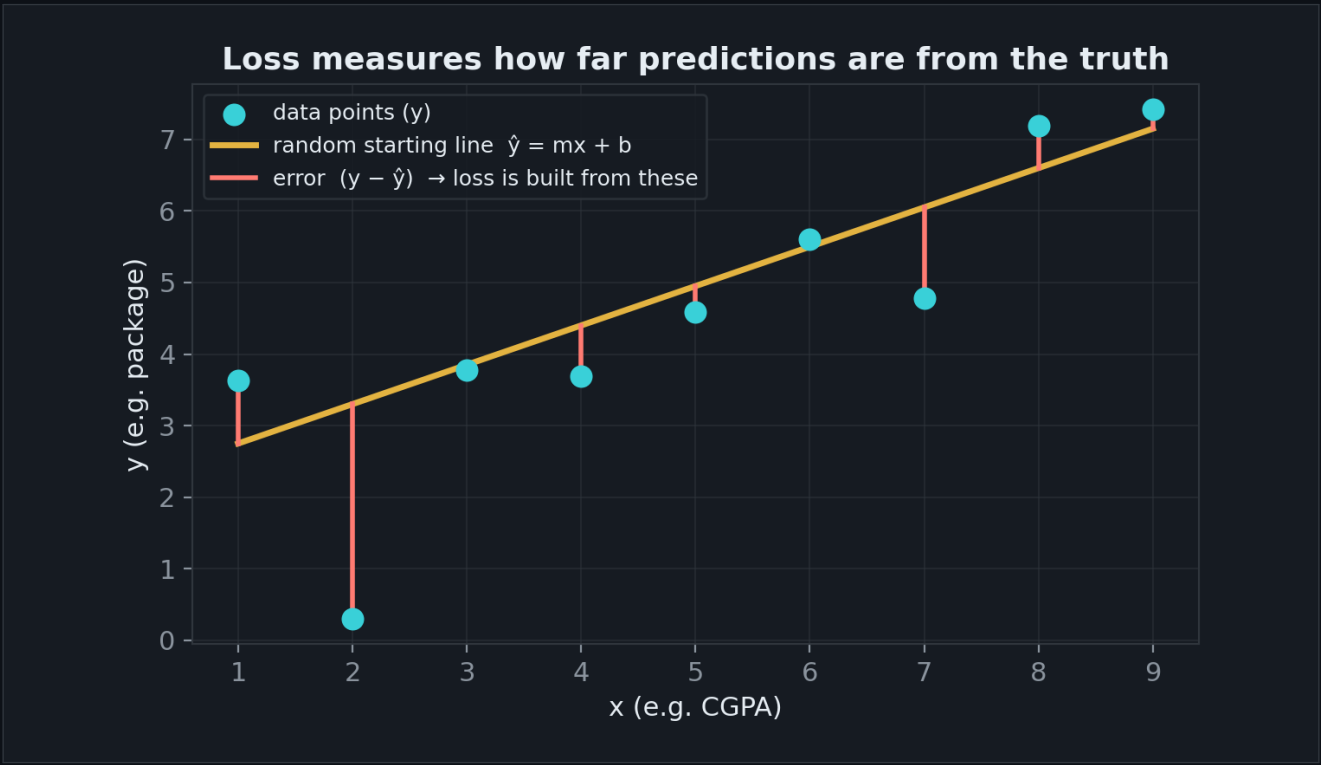


Figure 1.1 – The red segments are the errors (y − ŷ). The loss turns all of them into a single score.

If you tilt the line (change *m*) or slide it up and down (change *b*), the red segments get longer or shorter, so the loss goes up or down. **Training means: find the *m* and *b* that make the loss as small as possible.** The same idea holds in deep learning, except that a network has thousands or millions of weights instead of two.

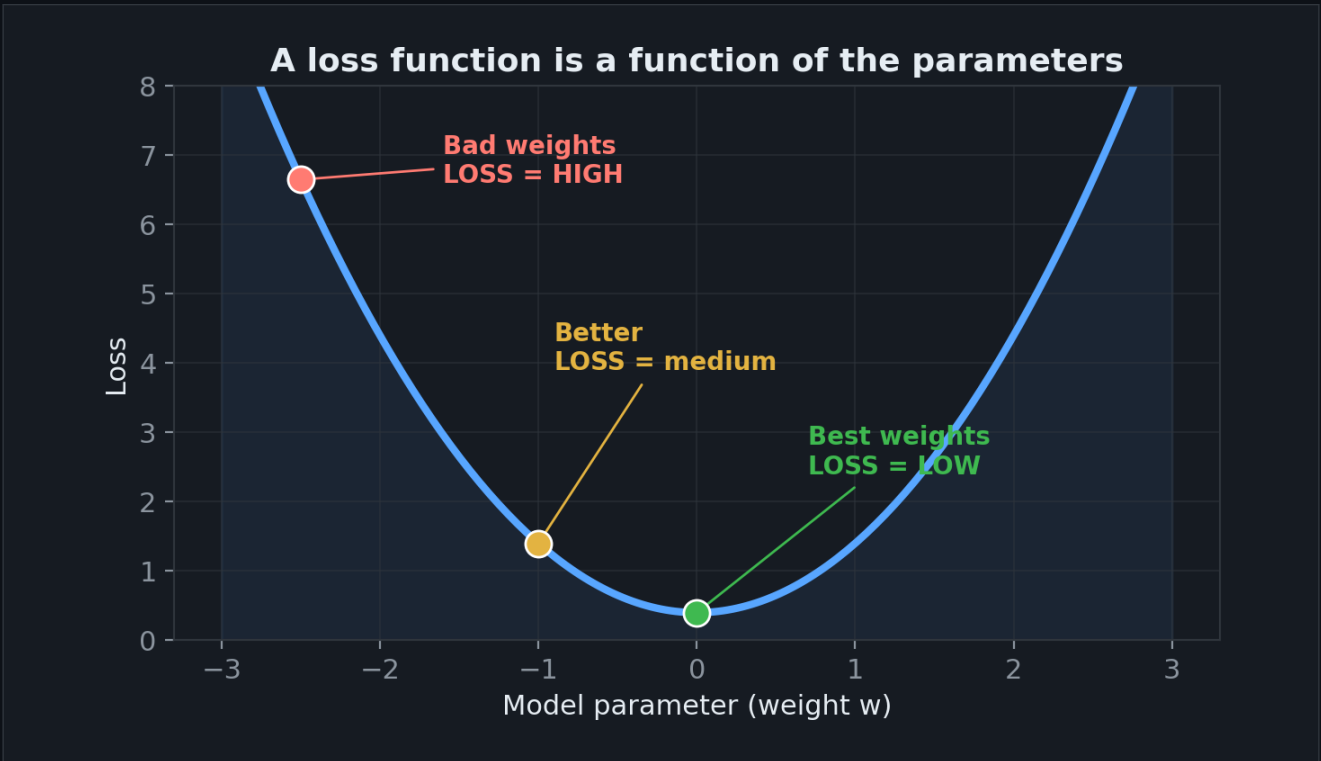


Figure 1.2 – Plot loss against one weight: you get a bowl. Training means walking to the bottom of the bowl.

1.2 Why is a loss function important?

The famous management thinker Peter Drucker said: “You can’t improve what you can’t measure.” A model that cannot measure its own mistakes cannot get better. The loss function is that measuring stick.

Think about how a model learns in classical machine learning:

1. Start with a **random** guess for the parameters (a random line).
2. Compute the **loss** of this guess: how far off are the predictions?
3. Use the loss to decide **how to change** the parameters (rotate the line a bit, move it a bit).
4. Compute the loss of the new line. Did it get better or worse? Adjust again.
5. Repeat until the loss cannot be reduced any more. The model is now **trained**.

> **FROM THE VIDEO** **The loss is the “eye” of the algorithm**
> The loss function tells the algorithm **where to go**. Which values to set for the parameters is decided by the loss. Take any ML algorithm and any problem, regression or classification: you will always find a loss function behind it, because without one the learning cannot happen.

1.3 The role of the loss in deep learning

Deep learning does the same thing with a neural network. The algorithm that trains the network is called **backpropagation** (the next topic after this tutorial). Here is a sneak preview so you can see where the loss fits. We use a tiny toy dataset: a student’s CGPA and IQ are the inputs, and the output we want to predict is the **package in LPA** (lakhs per annum).

Student	CGPA	IQ	Package y (LPA)
1	8.0	80	6.3
2	6.0	60	4.0
3	5.5	55	3.7
4	8.5	85	7.0

(The CGPA/IQ values are illustrative; the packages are the ones used in the lecture.)

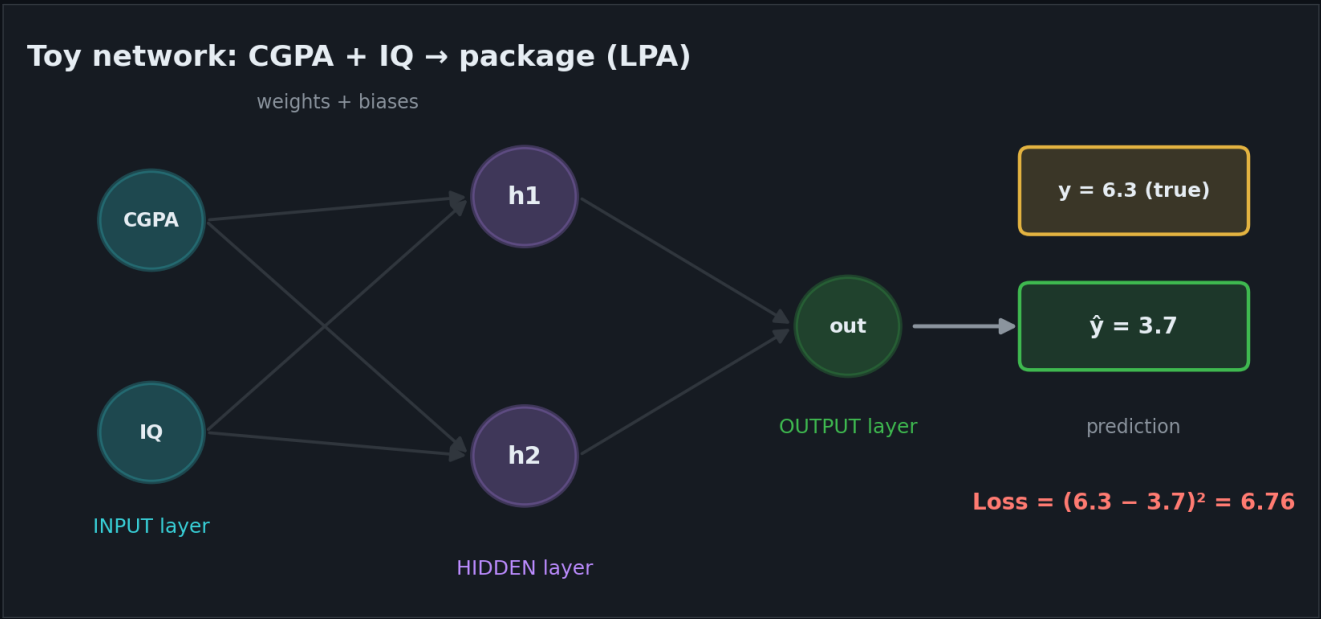


Figure 1.3 – A tiny network with 3 neurons: 2 in the hidden layer, 1 in the output layer. The loss compares ŷ with y.

The training loop is:

1. **Pick one student** (one row) and feed CGPA and IQ into the network. At the very start the weights and biases are **random**.
2. **Forward propagation:** the numbers flow through the layers (pure matrix maths) and the network outputs a prediction, say **ŷ = 3.7**.
3. **Choose a loss function.** This is a regression problem, so we use Mean Squared Error. Plug in the true value and the prediction. The result is one number telling us how badly the network did on this student.
4. **Go backwards:** use that number to adjust all weights and biases with **gradient descent**. (Details come in the next topic.)
5. Take the **next student** and repeat: forward pass, loss, update. Do this for the whole dataset, and then again for many **epochs** (one epoch = one full pass through the data).
6. The goal is to **minimise the loss**. When it is as low as it can go, the weights and biases you have are your final, trained weights.

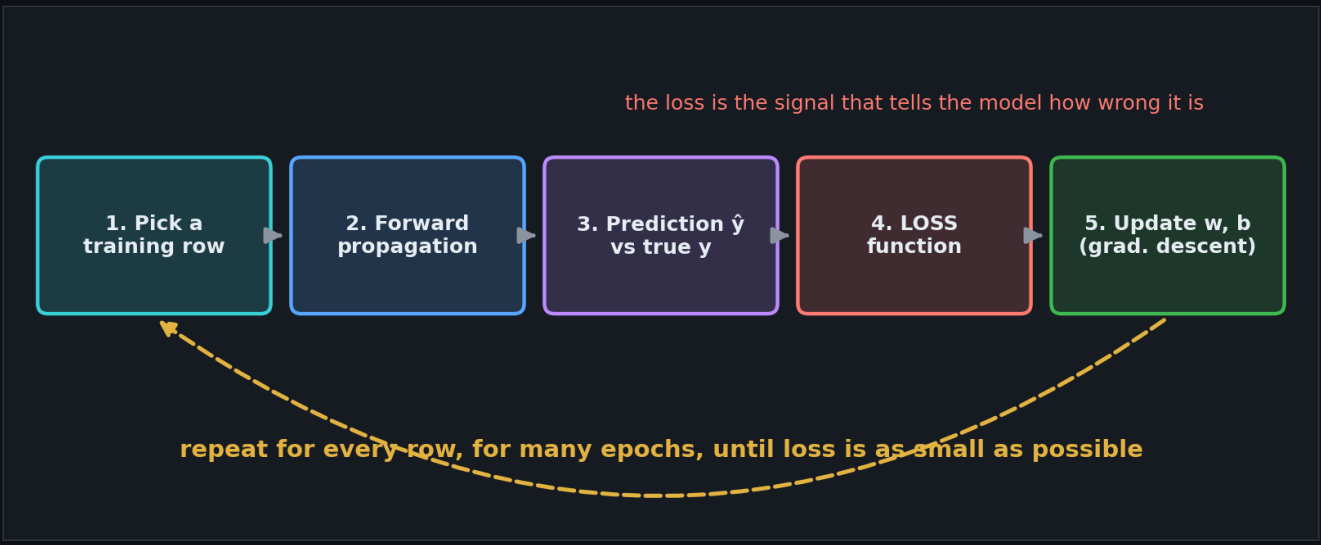


Figure 1.4 – The deep-learning training loop. The loss function sits at the heart of it.

A 60-second peek at gradient descent

Gradient descent is just “walk downhill on the loss bowl”. At the current weight w you measure the slope (the derivative of the loss). If the slope is positive you step left, if it is negative you step right. The step is scaled by a small number η (eta), the **learning rate**.

Gradient descent update rule

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial J}{\partial w}$$

Step	w	loss = w²	slope = 2w	new w = w − 0.1·slope
0	3.000	9.000	6.000	2.400
1	2.400	5.760	4.800	1.920
2	1.920	3.686	3.840	1.536
3	1.536	2.359	3.072	1.229

Notice how the loss falls each step: 9 → 5.76 → 3.69 → 2.36. The loss is the compass, the derivative gives the direction, and the learning rate sets the stride length.

1.4 Loss function vs Cost function

Many people use these two words as if they were the same. In careful language they are **different**, and interviewers love to ask about this.

	Loss function	Cost function
Computed over	**one** training example	a **whole batch** or the **entire** training set
Output	one number per row	one number for the whole data (the average of the losses)
Other names	error function	objective function
Symbol (usual)	L	J

Using the four students from the lecture (true y, prediction ŷ):

Student	y (true)	ŷ (predicted)	Loss = (y − ŷ)²
1	6.3	6.1	(6.3 − 6.1)² = 0.04
2	4.0	4.1	(4.0 − 4.1)² = 0.01
3	3.7	3.5	(3.7 − 3.5)² = 0.04
4	7.0	7.2	(7.0 − 7.2)² = 0.04
Cost J			**(0.04 + 0.01 + 0.04 + 0.04) / 4 = 0.0325**

Cost function for MSE over n points

$$J = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

TIP Interview-ready sentence

“A **loss** function is calculated for a **single** training example. A **cost** function is the **average** of the losses over the **whole** training set or batch. In practice, the cost is what we actually minimise.”

1.5 The map of loss functions

There is no single “best” loss. The loss you choose depends on **what problem you are solving**. Here is the full map. In this tutorial we visit every box.

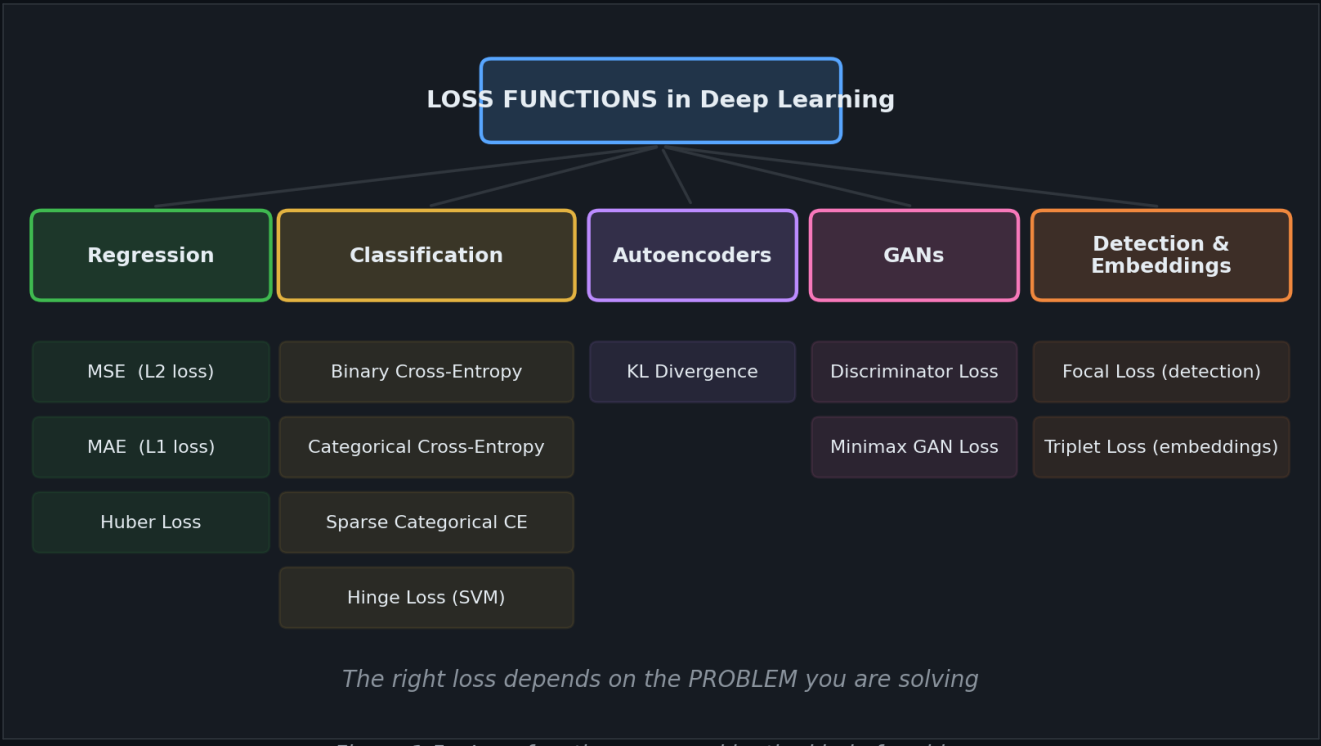


Figure 1.5 – Loss functions grouped by the kind of problem.

Problem type	Typical losses	Where you meet it
Regression (predict a number)	MSE, MAE, Huber	house price, package, temperature
Classification (predict a class)	Binary CE, Categorical CE, Sparse CE, Hinge	spam / not spam, digit 0–9, SVM
Autoencoders	KL Divergence	VAE, compression, generating data
GANs	Discriminator loss, Minimax GAN loss	image generation
Object detection	Focal loss	finding objects when most pixels are background
Embeddings	Triplet loss	face recognition, similarity search

NOTE You can also invent your own

Researchers create new losses all the time, and in Keras you can write your own loss function in a few lines (we do this in Part 5). Every loss has advantages and disadvantages. If you pick the wrong one, you will not reach the best solution, so choose according to your data and goals.

KEY IDEA **Summary of Part 1**

• Loss = one number that scores the model’s mistake. • It depends on the parameters, so we can lower it by changing them. • It guides gradient descent. • Loss is per example, cost is the average over many. • Different problems need different losses.

PART 2 • LEVEL 2 · INTERMEDIATE

Regression Losses: MSE, MAE and Huber

Regression means predicting a **number** (a package, a price, a temperature). Here you meet the three most used regression losses, learn why squaring matters, what outliers do, and how to pick between them.

2.0 Setting up a regression network

For every regression loss in this part we use the same setup as before: two inputs (CGPA, IQ), one hidden layer, and a single output neuron that predicts the package. One rule is crucial:

WATCH OUT The last neuron must be linear

For regression the output neuron needs a **linear** activation (i.e. no squashing). The hidden layers can use ReLU, Sigmoid, etc., it makes no difference, but the last one must be linear so the network can output any real number. Only then can you sensibly apply MSE (and MAE/Huber).

2.1 Mean Squared Error (MSE)

MSE is the most common loss in regression. Other names: **squared loss** and **L2 loss**. All three names are valid. You most likely met it in linear regression.

The formula

MSE loss for ONE example

$$L(y, \hat{y}) = (y - \hat{y})^2$$

Read it in plain English: **true value minus predicted value, then square it.** For a whole dataset of n points we average:

MSE cost for n examples

$$J = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

How the training uses it – step by step

1. Insert row 1 into the network and run forward propagation to get ŷ.
2. Compute the loss. For student 1, y = 6.3 and ŷ = 6.1, so loss = $(6.3 - 6.1)^2$ = **0.04**.
3. Go back and update weights and biases using this value (gradient descent).
4. Insert row 2 and repeat. Keep going. The idea is to **minimise** the loss.

Why do we square? (The cancellation problem)

A natural first thought is: why not simply use $y - \hat{y}$? Because errors have signs. Some predictions are too high (negative error), some are too low (positive error). If you add them up, **they cancel each other** and a bad model can look perfect.

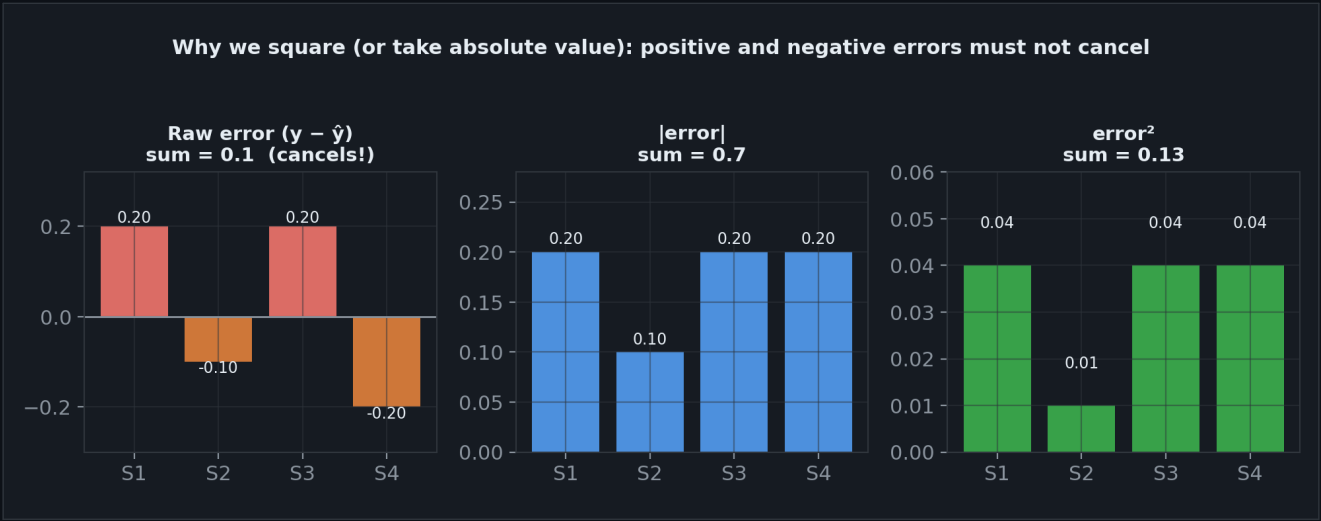


Figure 2.1 – Raw errors sum to 0.1 even though every prediction is off. Squaring (or absolute value) makes every error count.

With our four students the raw errors are 0.2, −0.1, 0.2, −0.2 and their sum is just **0.1**. After squaring they sum to **0.13**, a true picture of the mistakes. Taking the absolute value would also work: that is another loss, MAE, coming next.

What squaring does: big errors are punished much more

Squaring makes the loss **quadratic**. Look at what happens to errors of different sizes:

Error size	1	2	3	4	5	10
MSE (error²)	1	4	9	16	25	100
MAE (|error|)	1	2	3	4	5	10



Figure 2.2 – MSE is a parabola (a gentle bowl near zero, steep far away); MAE is a V-shape.

Doubling the error from 2 to 4 quadruples the penalty from 4 to 16. The farther a point is from the truth, the more its error is **magnified**.

KEY IDEA What this means for a regression line

Imagine fitting a line to points. A point **close** to the line has a small error, so it nudges the line only a little. A point **far** from the line has a huge squared error, so it pulls the line hard, causing a big rotation. In a neural network the same thing happens to the weights: far-away points cause **drastic** updates; nearby points cause **tiny** updates. The far points are being "punished".

Advantages of MSE

- **Easy to interpret.** Just look at the formula and you see how the error is computed.
- **Always differentiable.** The curve is a smooth bowl with no sharp corners, so gradient descent can compute its slope everywhere easily.
- **Only one local minimum.** There is exactly one best set of weights for a linear model, so gradient descent cannot get trapped in a false valley (Figure 2.3).

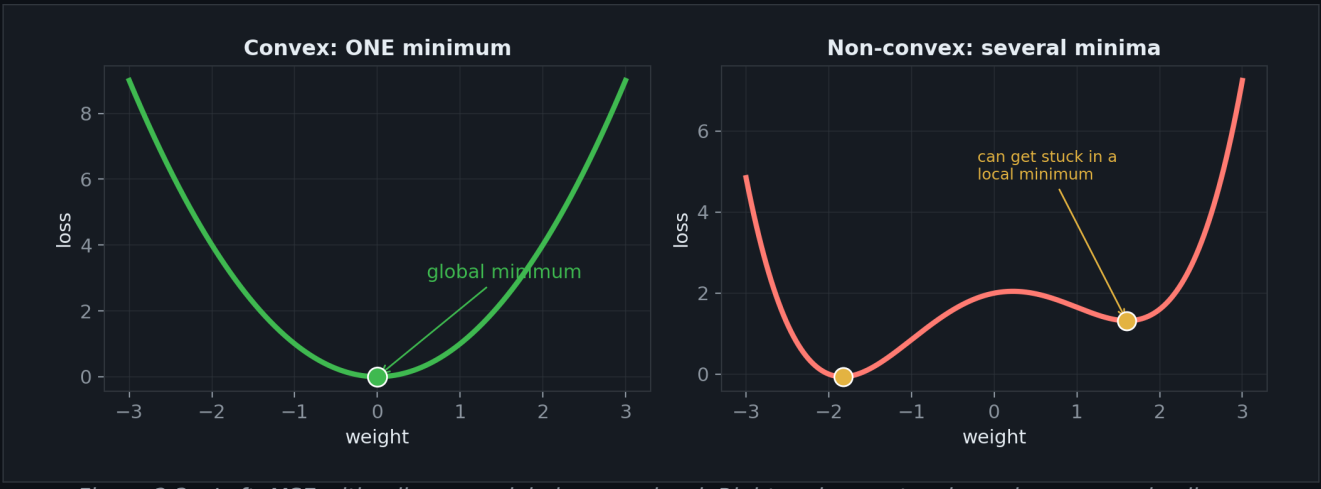


Figure 2.3 – Left: MSE with a linear model gives one bowl. Right: a deep network can have several valleys.

Disadvantages of MSE

1. The unit is squared. If y is in LPA, the loss is in LPA^2. An MSE of 5 does *not* mean "5 LPA off"; it means 5 LPA^2. Taking the square root brings you back to the original unit. That is called **RMSE**:

Root Mean Squared Error (same unit as y)

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

2. It is not robust to outliers. An outlier is a point that does not follow the pattern. Because errors are squared, one outlier can dominate everything.

EXAMPLE Outlier example (from the lecture)

A student has CGPA 8.1 and IQ 100 and got a package of **50 LPA**, but the model predicts about **8 LPA**. The error is 42, so the MSE term is 42^2 = **1764**. Compare that with the other four students whose squared errors are only about 0.04. This one student becomes the main teacher: the weights get updated violently to reduce that 1764, and the model gets thrown off for everyone else.

Data	MSE	MAE
4 normal students	0.0325	0.175
+ 1 outlier (error 42)	352.8	8.54
Increase	x 10,856	x 49

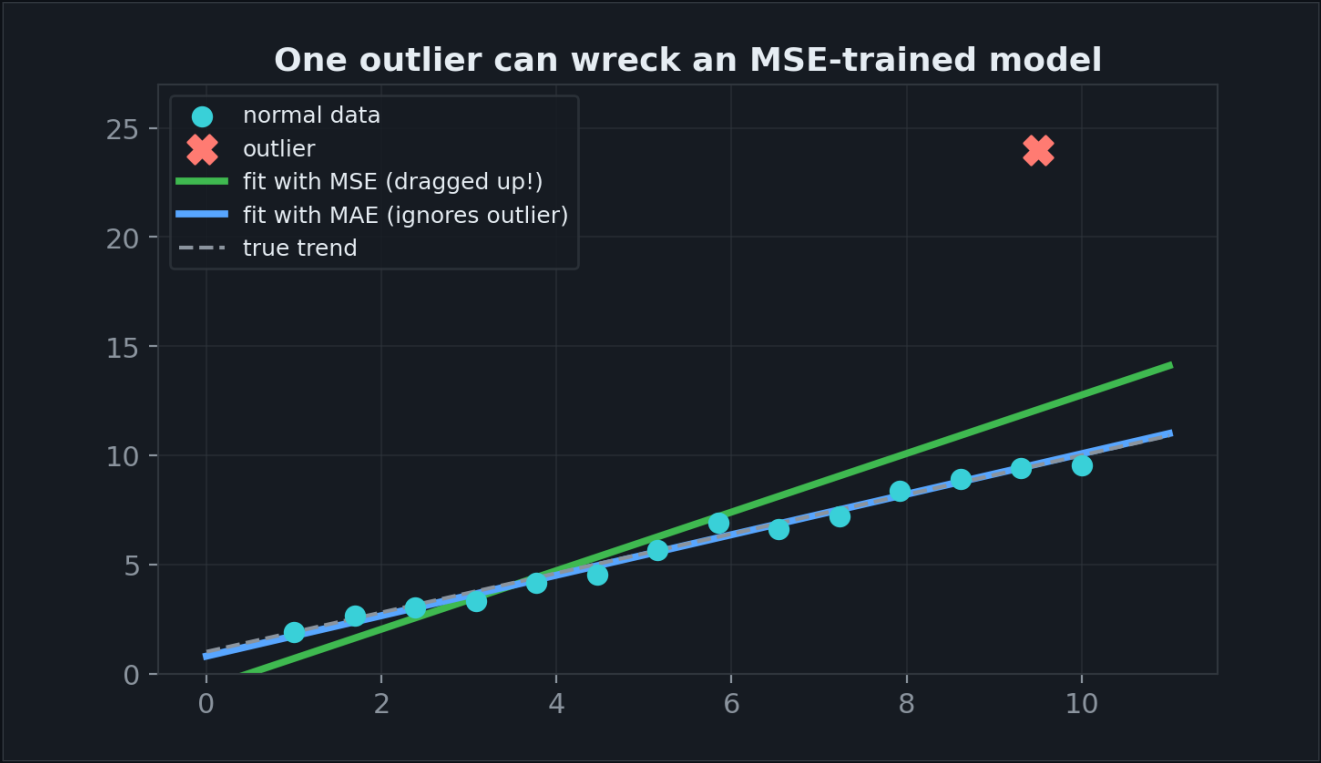


Figure 2.4 – With one outlier, the MSE line (green) is dragged away from the true trend. The MAE line (blue) stays on the data.

The gradient of MSE (what the network actually feels)

Derivative of the MSE loss with respect to the prediction

$$\frac{\partial L}{\partial \hat{y}} = -2\,(y - \hat{y})$$

The “push” on the weights is proportional to the error itself. Error 42 gives a push of 84; error 0.2 gives a push of 0.4. That is the mathematical reason big errors dominate.

MSE in Keras

Remember: linear output neuron, then choose `mse` when compiling. This is exactly what is done in the GRE-admission prediction example of the video series.

Python (Keras / TensorFlow)

```python
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense

model = Sequential([
    Dense(8, activation="relu", input_shape=(2,)),   # hidden layer (CGPA, IQ)
    Dense(1, activation="linear")                    # output: package in LPA  <-- must be linear
])
model.compile(optimizer="adam", loss="mean_squared_error")   # or loss="mse"
model.fit(X_train, y_train, epochs=100, validation_split=0.2)
```

TIP Use MSE when…
your data has **no (or few) outliers**, you want big mistakes punished strongly, and you want smooth gradients. It is the default choice for regression.

2.2 Mean Absolute Error (MAE)

MAE is the second regression loss. Other names: **L1 loss**. It fixes most of MSE's problems by replacing the square with an **absolute value** ("mod"): the same formula, with $|\cdot|$ in place of $(\cdot)^2$.

MAE loss for ONE example

$$L(y, \hat{y}) = |y - \hat{y}|$$

MAE cost for n examples

$$J = \frac{1}{n} \sum_{i=1}^{n} |y_i - \hat{y}_i|$$

The cost formula for MSE and MAE are identical in structure: average, over the n points, of the per-example loss. Only the per-example loss changes.

Worked example

Student	y	ŷ	|y − ŷ|	(y − ŷ)²
1	6.3	6.1	0.2	0.04
2	4.0	4.1	0.1	0.01
3	3.7	3.5	0.2	0.04
4	7.0	7.2	0.2	0.04
Average			**0.7 / 4 = 0.175**	**0.13 / 4 = 0.0325**

MAE = 0.175 LPA. This number can be read directly: **on average the model is 0.175 LPA off.**

Advantages of MAE

- **Intuitive.** Subtract and take the absolute value. Nothing else.
- **Same unit as y.** If y is in LPA, the error is in LPA: "my error is 2 LPA". You understand it at a glance.
- **Robust to outliers.** Without a square, the error grows linearly: 2 stays 2, 4 stays 4 (not 16). An outlier is not punished terribly, so the line stays near the bulk of the data (blue line in Figure 2.4).

Disadvantage of MAE

Its graph is a V with a **sharp corner at zero error**, so it is **not differentiable** there. Gradient descent relies on derivatives, so you need **sub-gradients** (use −1 or +1 depending on the sign, and pick 0 at the corner). This is a bit more tedious and the constant-size gradient can also make training noisy near the minimum, because the push does not shrink as the error gets small.

Sub-gradient of MAE (only the sign of the error matters)

$$\frac{\partial L}{\partial \hat{y}} = -\operatorname{sign}(y - \hat{y})$$

Python (Keras / TensorFlow)

```
model.compile(optimizer="adam", loss="mean_absolute_error")   # or loss="mae"
```

TIP Use MAE when…
your data **contains outliers** and you want the model to ignore them rather than chase them. Rule of thumb from the lecture: **no outliers → MSE, outliers → MAE**.

2.3 Huber Loss

MSE reacts *too strongly* to outliers. MAE treats outliers *exactly like* normal points. What if you want something in between?

Imagine 25% of your points are “outliers”. With that many, they are not really outliers any more: they are a part of your data. MSE will let them pull the line to the middle, MAE will lean entirely towards the other 75%. You want a line **mostly** towards the 75% and **a bit** towards the 25%. That compromise is Huber loss.

The formula

Huber loss uses a threshold **δ** (delta). It behaves like MSE for small errors and like MAE for large errors:

Small error (not an outlier): quadratic, like MSE

$$L_\delta = \frac{1}{2}(y - \hat{y})^2 \qquad \text{if} \quad |y - \hat{y}| \le \delta$$

Large error (outlier): linear, like MAE

$$L_\delta = \delta\,|y - \hat{y}| - \frac{1}{2}\delta^2 \qquad \text{otherwise}$$

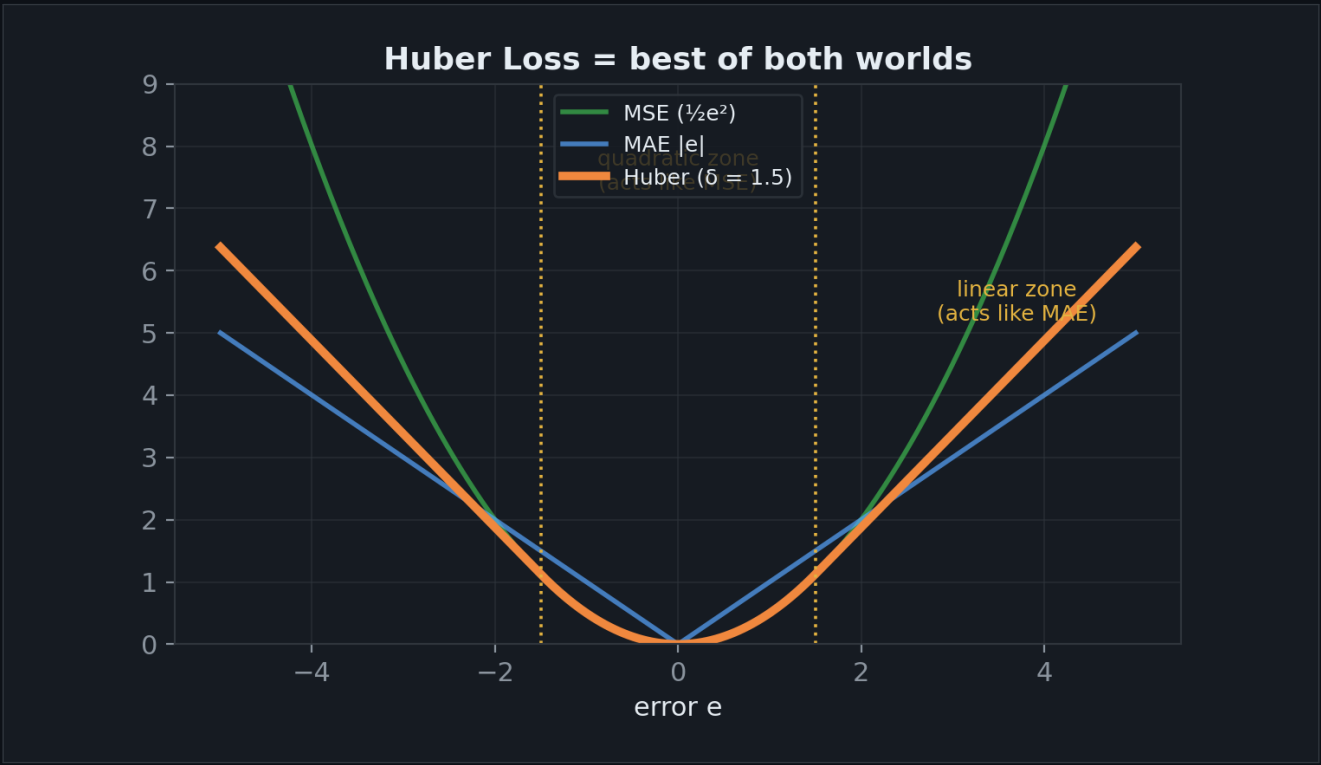


Figure 2.5 – The orange curve is smooth near zero like MSE, then continues in a straight line like MAE.

What is δ? It is a **hyperparameter**: the error size at which we call a point an “outlier”. You tune it and watch the performance. A small δ makes Huber behave like MAE; a very large δ makes it behave like MSE.

The second formula subtracts ½δ² so that the two pieces join *smoothly* at |error| = δ with no jump.

Worked example with δ = 1

Error e	MSE (e²)	MAE (|e|)	Huber (δ = 1)	Which branch?
0.2	0.04	0.2	0.02	quadratic
1	1	1	0.5	quadratic
3	9	3	2.5	linear
42	1764	42	41.5	linear

For the outlier (error 42), MSE gives 1764, Huber gives only 41.5. For small errors, Huber is smooth just like MSE.

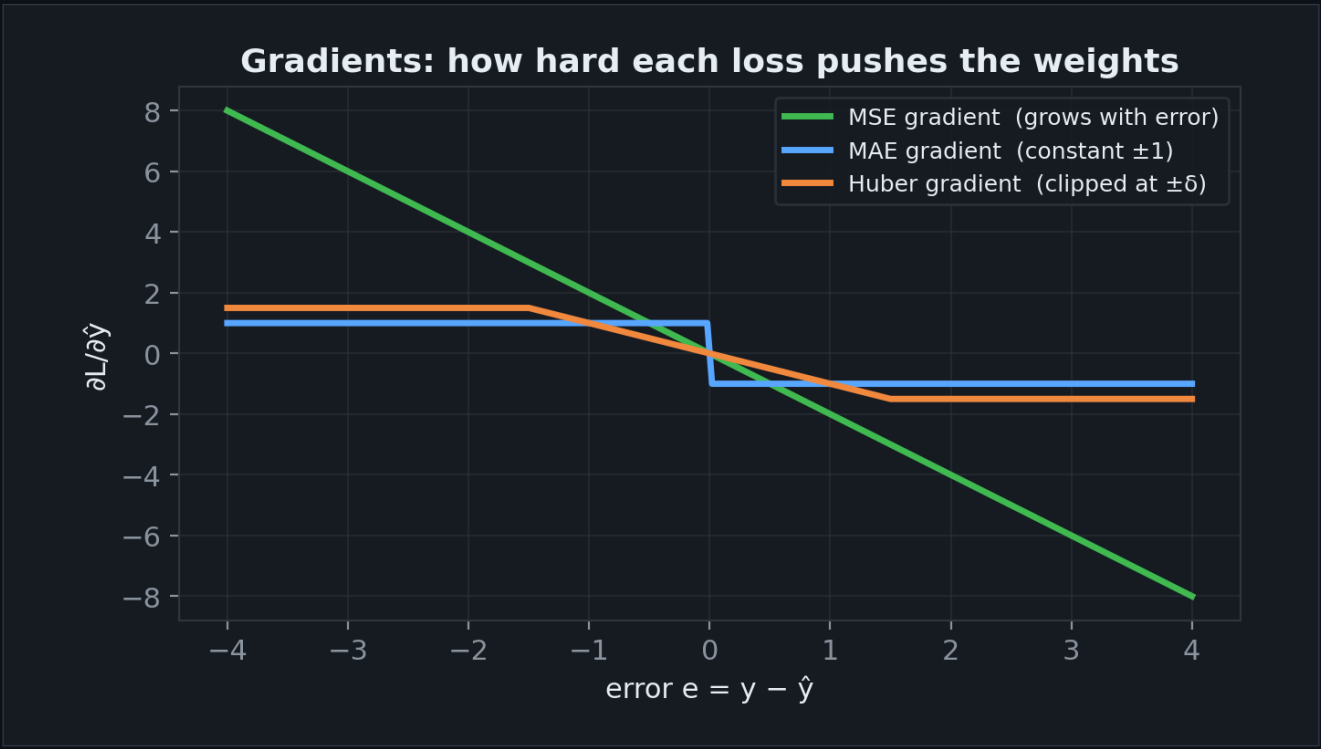


Figure 2.6 – Gradients: MSE grows without limit, MAE is a constant ±1, Huber grows then clips. That clipping is what protects training from outliers.

PRO LEVEL Smooth AND robust

Huber is differentiable everywhere (MAE is not) and caps the gradient size for outliers (MSE does not). That is why it is popular in reinforcement learning (DQN) and robust regression.

Python (Keras / TensorFlow)

```
import tensorflow as tf
model.compile(optimizer="adam", loss=tf.keras.losses.Huber(delta=1.0))
```

2.4 MSE vs MAE vs Huber at a glance

	MSE (L2)	MAE (L1)	Huber
Formula	$(y - \hat{y})^2$	$\lvert y - \hat{y} \rvert$	quadratic if $\lvert e \rvert \le \delta$, else linear
Unit of the error	squared (LPA^2)	same as y (LPA)	depends on branch
Outliers	very sensitive	robust	robust (tunable via δ)
Differentiable?	yes, everywhere	no, at 0	yes, everywhere
Local minima (linear model)	one	one	one
Gradient size	grows with error	constant	grows, then capped
Best for	clean data	data with many outliers	a mix of normal points and outliers
Keras name	mse	mae	Huber(delta=...)

KEY IDEA **Summary of Part 2**

• Regression → linear last neuron. • **MSE**: squares the error, punishes outliers, smooth, unit squared. • **MAE**: absolute error, robust, same unit, sharp corner. • **Huber**: MSE for small errors, MAE for big ones, controlled by δ.

PART 3 • LEVEL 3 · UPPER-INTERMEDIATE

Classification Losses: Cross-Entropy and Hinge

Classification means predicting a **category**. The model outputs **probabilities**, so we need losses that compare probabilities with the true label. Here you learn Binary Cross-Entropy, Categorical Cross-Entropy, Sparse Categorical Cross-Entropy and Hinge loss.

3.0 Before we start: probabilities and the idea of “surprise”

In classification the network ends with an activation that turns raw scores into **probabilities** between 0 and 1. The loss then asks: *how much probability did you give to the correct answer?*

ANALOGY The weather forecaster
Forecaster A says “99% chance of rain” and it stays dry: very embarrassing, a huge penalty. Forecaster B says “51% rain” and it stays dry: a small penalty. Forecaster C says “99% rain” and it rains: almost no penalty. **Cross-entropy loss is that penalty system**: it rewards confident correct answers and brutally punishes confident wrong ones. Mathematically the penalty is **−log(probability given to the truth)**.

Why the logarithm? If the model gives probability 1 to the right answer, −log(1) = 0 (no loss). As the probability given to the right answer shrinks towards 0, −log(p) shoots up towards infinity. That is exactly the behaviour we want:

Probability given to the TRUE class	1.0	0.9	0.7	0.5	0.25	0.1	0.01
Loss = −ln(p)	-0.000	0.105	0.357	0.693	1.386	2.303	4.605

NOTE Which logarithm?
The lecture’s hand calculations (0.13 and 0.12 below) use $\log_{10}$. Deep-learning libraries (Keras, PyTorch) use the **natural log (ln)**. The shape is the same and only the scale differs (ln = 2.303 × $\log_{10}$). I show both so you can match the video *and* what Keras prints.

3.1 Binary Cross-Entropy (BCE)

Use it when there are exactly **two classes**: Yes/No, spam/not spam, placed/not placed. It is the same loss used in **logistic regression**. It is also called **Log Loss**.

The network for BCE

WATCH OUT Rule: one output neuron with a SIGMOID activation
Hidden layers may use ReLU or Sigmoid, but the output layer must have **one neuron** with **Sigmoid**, because BCE needs a probability between 0 and 1.

Sigmoid activation (squashes any number into 0…1)

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

The formula

BCE loss for ONE example (y = true label 0 or 1, ŷ = predicted probability)

$$L = -\,y\log(\hat{y}) \;-\; (1-y)\log(1-\hat{y})$$

This long formula is really two tiny formulas wearing one coat. Since y is either 1 or 0, one of the two terms always vanishes:

If the true label is 1: only the first term survives

$$y = 1: \quad L = -\log(\hat{y})$$

If the true label is 0: only the second term survives

$$y = 0: \quad L = -\log(1-\hat{y})$$

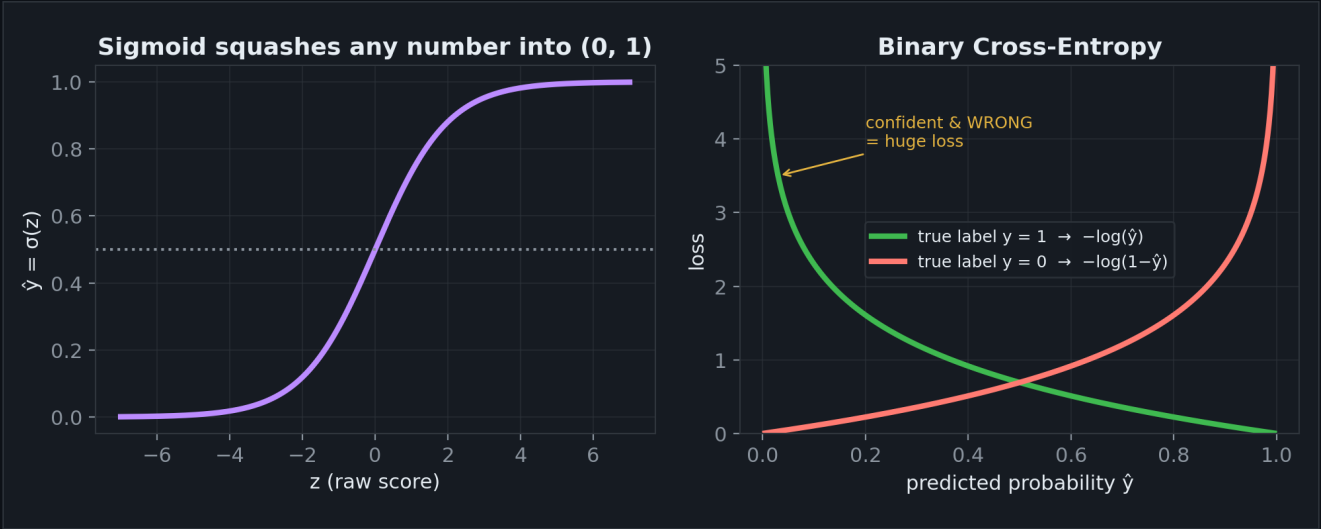


Figure 3.1 – Left: the sigmoid. Right: BCE loss. When the truth is 1 you want ŷ near 1; when the truth is 0 you want ŷ near 0.

And for a dataset of m examples, the cost is the average (with a minus sign outside):

BCE cost over m examples

$$J = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i\log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

Worked example (placement prediction)

Dataset: students' CGPA and IQ, and the target **placed (1) / not placed (0)**. We feed student 1 through the network and the Sigmoid output neuron says **ŷ = 0.73**. The true label is y = 1.

	Student 1	Student 2
True label y	1	0
Predicted ŷ (sigmoid)	0.73	0.25
Which term is alive?	$-1 \cdot \log(0.73)$	$-(1-0) \cdot \log(1 - 0.25) = -\log(0.75)$
Loss with $\log_{10}$ (video)	0.14	0.12
Loss with ln (Keras)	0.315	0.288

Student 1 is predicted placed with 73% probability and truly is placed → a smallish loss. Student 2 is truly not placed and the model said 25% placed (so 75% not placed) → also a smallish loss. Cost over the two students (ln): (0.315 + 0.288) / 2 = **0.301**.

EXAMPLE See the punishment for being confidently wrong

True label y = 1. Prediction 0.9 → loss 0.105. Prediction 0.5 → loss 0.693. Prediction 0.1 → loss 2.303. Prediction 0.01 → loss 4.605. The loss rockets up as the model becomes confidently wrong. This is the signal that forces the network to fix its worst mistakes quickly.

Where does this formula come from? (optional, a taste of the maths)

It comes from **Maximum Likelihood**. A model that assigns probabilities ŷ to the labels has a *likelihood* of the observed data equal to the product of the probabilities given to the correct answers. We want that likelihood as big as possible. Products are awkward and numerically tiny, so we take the log (turning products into sums) and flip the sign (so "maximise" becomes "minimise"). The result is exactly BCE:

From likelihood to BCE

$$-\log\left[\prod_i \hat{y}_i^{y_i}(1-\hat{y}_i)^{1-y_i}\right] = -\sum_i [y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)]$$

The full derivation is in the logistic-regression lesson of the "100 Days of Machine Learning" playlist. You do not need it to *use* BCE; Keras handles it behind the scenes.

Advantages and disadvantages

- ✓ The function is **differentiable**, so gradient descent works smoothly.
- ✓ Gives **very strong gradients when the model is confidently wrong** (far better than MSE for classification).
- ✗ Not intuitive: from the formula alone it is hard to "see" the loss, which is why the diagrams above help.
- ✗ In a deep network the overall loss surface can contain **multiple local minima** (the network is non-linear). For plain logistic regression BCE is actually convex, but in deep networks the lecture's warning applies.

Python (Keras / TensorFlow)

```
model = Sequential([
    Dense(8, activation="relu", input_shape=(2,)),
    Dense(1, activation="sigmoid")          # ONE neuron + sigmoid  <-- required
])
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

3.2 Categorical Cross-Entropy (CCE)

Use it when there are **three or more classes**. Example: placement outcome = *Yes / No / Maybe*, or handwritten digits 0–9. It is also the loss of **softmax regression**.

The network for CCE changes a little

> **WATCH OUT** **Rule: k output neurons + SOFTMAX**
> Put exactly **as many neurons in the output layer as there are classes** (3 classes → 3 neurons; the 10 digits of MNIST → 10 neurons) and use the **Softmax** activation on all of them.

Softmax: turning scores into probabilities

Each output neuron computes a raw score z (the weighted sum). Softmax converts the set of scores into probabilities:

Softmax over k classes

$$\hat{y}_j = \text{softmax}(z)_j = \frac{e^{z_j}}{\sum_{c=1}^{k} e^{z_c}}$$

For three classes (Yes, No, Maybe) the three outputs are $e^{z1}/(e^{z1}+e^{z2}+e^{z3})$, and similarly for z2 and z3. Take scores 2.0, 1.0, 0.1: e^{2}=7.389, e^{1}=2.718, $e^{0.1}$=1.105, total = 11.212. The probabilities are **0.659, 0.242, 0.099**.

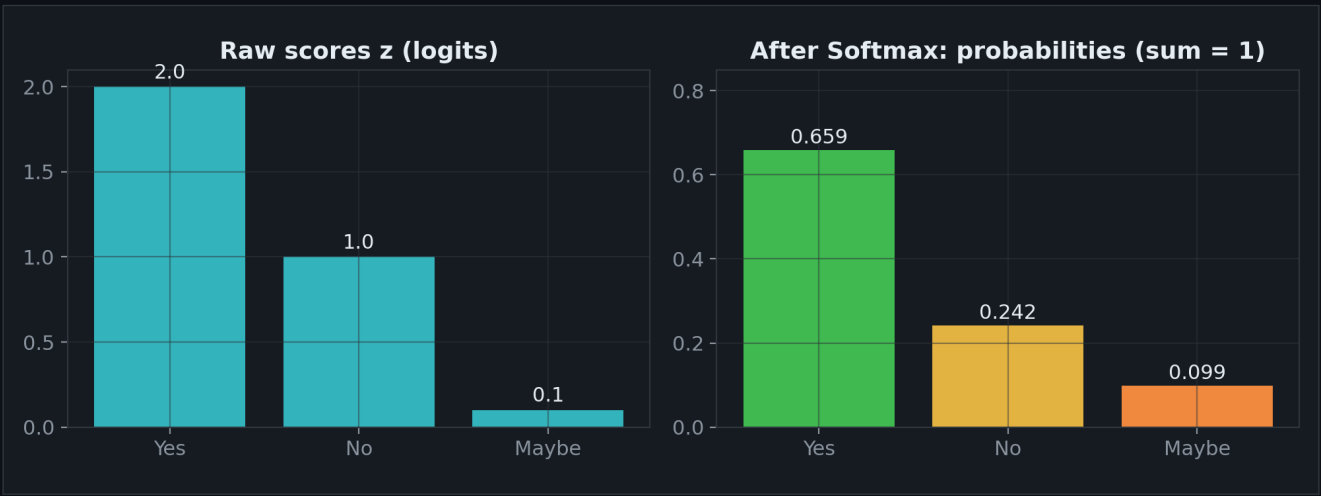


Figure 3.2 – Softmax keeps the ranking of the scores but forces the outputs to be between 0 and 1 and to add up to exactly 1.

Always true: every softmax output lies between 0 and 1, and **all outputs add up to 1**.

One-hot encoding the labels

For CCE we first **one-hot encode** the target column: one slot per class, with a 1 in the true class and 0 elsewhere.

Original label	One-hot vector
Yes	[1, 0, 0]
No	[0, 1, 0]
Maybe	[0, 0, 1]

The formula

CCE loss for ONE example (k = number of classes)

$$L = -\sum_{j=1}^{k} y_j \log(\hat{y}_j)$$

For our 3 classes, write it out:

Expanded for three classes

$$L = -(y_1 \log \hat{y}_1 + y_2 \log \hat{y}_2 + y_3 \log \hat{y}_3)$$

Magic simplification: because the label is one-hot, **all terms but one are multiplied by 0**. Only the log of the probability given to the **true class** remains: loss = $-\log(\hat{y}_{\text{true}})$. This is the same "weather forecaster" penalty from before.

Worked examples

Student 1 (CGPA 8, IQ 80), true class **Yes** = [1, 0, 0]. The network outputs [0.2, 0.3, 0.5]. Only $y_1 = 1$ survives:

loss = $-(1 \cdot \log 0.2 + 0 \cdot \log 0.3 + 0 \cdot \log 0.5) = -\log(0.2) =$ **1.609** (ln) or 0.699 ($\log_{10}$).

Student 2 (CGPA 6, IQ 60), true class **No** = [0, 1, 0]. The network outputs [0.3, 0.6, 0.1]. Only $y_2 = 1$ survives:

loss = $-\log(0.6) =$ **0.511** (ln) or 0.222 ($\log_{10}$).



Figure 3.3 – Student 1 gave only 0.2 to the true class → high loss. Student 2 gave 0.6 → low loss.

Cost over many examples n (average of all per-example losses):

CCE cost

$$J = -\frac{1}{n} \sum_{i=1}^{n} \sum_{j=1}^{k} y_{ij} \log(\hat{y}_{ij})$$

For our two students: (1.609 + 0.511) / 2 = **1.060**.

Python (Keras / TensorFlow)

```python
from tensorflow.keras.utils import to_categorical
y_train_oh = to_categorical(y_train, num_classes=3)     # one-hot encode labels

model = Sequential([
    Dense(8, activation="relu", input_shape=(2,)),
    Dense(3, activation="softmax")                      # 3 classes -> 3 neurons + softmax
])
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
# MNIST example: Dense(10, activation="softmax")  (digits 0..9)
```

3.3 Sparse Categorical Cross-Entropy

You will often see `sparse_categorical_crossentropy` (the MNIST example used it). Here is the surprise: it is **the same loss** as Categorical Cross-Entropy. Only the **way the labels are stored** changes.

	Categorical CE	Sparse Categorical CE
Label format	one-hot vectors: [1,0,0], [0,1,0], [0,0,1]	integers: 1, 2, 3 (Keras counts from 0, 1, 2)
One-hot step needed?	yes	**no**
Output layer	k neurons + softmax	k neurons + softmax (identical)
Loss value	identical	identical
Speed / memory	slower with many classes	faster, less memory with many classes

How the loss is computed: look at the integer label, grab **only that neuron's probability**, ignore the others, and take −log of it.

True label (int)	Predicted probabilities	Neuron used	Loss (ln)
0 (Yes)	[0.1, 0.5, 0.4]	1st → 0.1	−ln(0.1) = 2.303
1 (No)	[0.2, 0.5, 0.3]	2nd → 0.5	−ln(0.5) = 0.693
2 (Maybe)	[0.6, 0.2, 0.2]	3rd → 0.2	−ln(0.2) = 1.609

Sparse categorical CE (c = index of the true class)

$$L = -\log(\hat{y}_c) \qquad c = \text{index of the true class}$$

Why is it faster? You never compute the logs of the other k − 1 probabilities, and you never build giant one-hot vectors. With 10 classes the saving is small. With 50,000 classes (e.g. a vocabulary in language models) it is large.

TIP Quick rule

few classes and you already have one-hot labels → Categorical CE. Many classes, or labels as integers → **Sparse** Categorical CE. Same maths, less work.

Python (Keras / TensorFlow)

```python
# labels are plain integers like 0,1,2,...  (no to_categorical needed)
model = Sequential([
    Dense(128, activation="relu", input_shape=(784,)),
    Dense(10, activation="softmax")                    # MNIST: 10 neurons + softmax
])
model.compile(optimizer="adam", loss="sparse_categorical_crossentropy", metrics=["accuracy"])
```

3.4 Hinge Loss (the SVM loss)

The video lists Hinge Loss in the map as the loss of **SVMs (Support Vector Machines)** and also usable in neural networks. Instead of probabilities it cares about **margins**.

ANALOGY **Passing mark with a safety buffer**

Imagine a pass mark where you must clear it *by a margin* to be completely safe. Once a point is correctly classified **with enough margin**, hinge loss says “perfect, zero loss, I stop caring”. Only points that are wrong *or too close to the boundary* are penalised.

Labels are encoded as **−1 and +1** (not 0/1), and the model outputs a raw score f(x) (no sigmoid).

Hinge loss

$$L = \max(0,\ 1 - y \cdot f(x)) \qquad y \in \{-1,\ +1\}$$

The product y·f(x) is the **margin**: positive means the correct side, ≥ 1 means safely on the correct side.

True y	Score f(x)	Margin y·f	Loss max(0, 1 − margin)	Meaning
+1	2.0	2.0	0	correct with room to spare
+1	0.3	0.3	0.7	correct, but inside the margin
−1	0.4	−0.4	1.4	wrong side
−1	−1.5	1.5	0	correct with room to spare

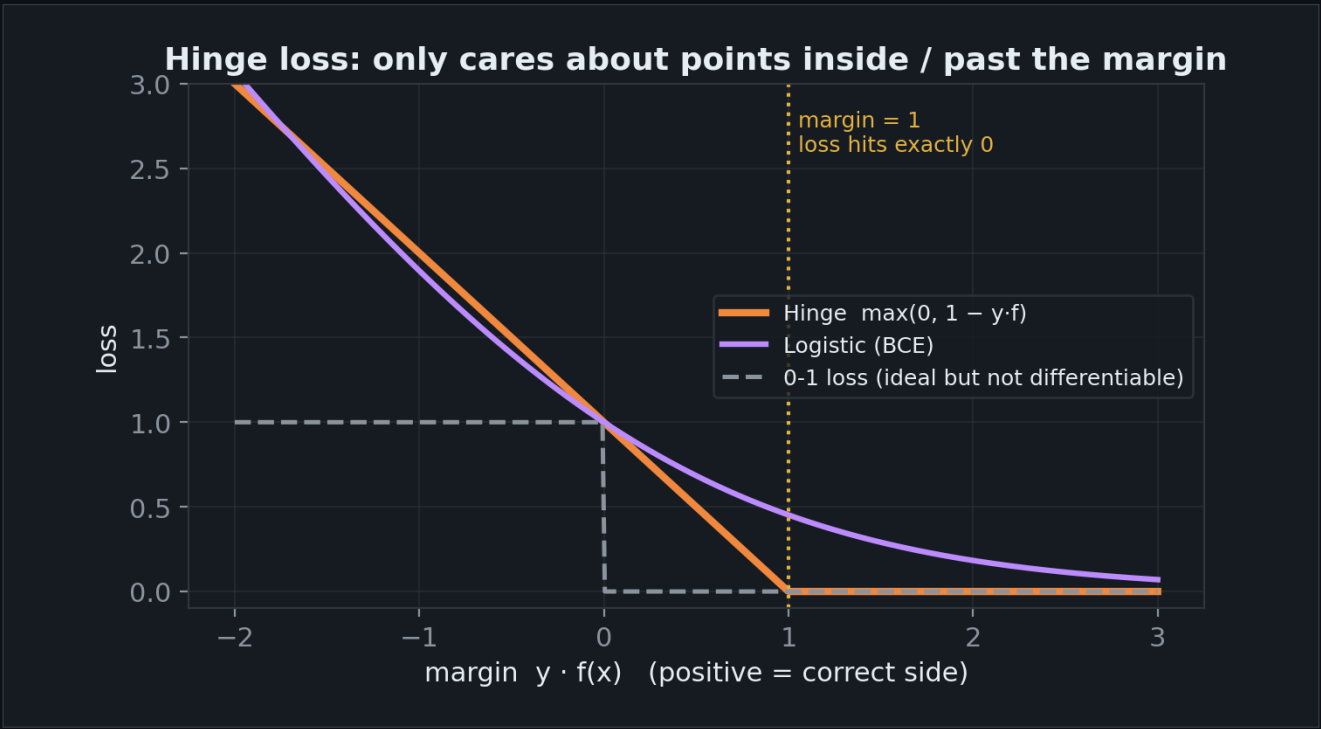


Figure 3.4 – Hinge loss is flat at 0 beyond the margin, then rises linearly. It is a convex stand-in for the ideal-but-untrainable 0-1 loss.

Python (Keras / TensorFlow)

```
model.add(Dense(1, activation="linear"))              # raw score
model.compile(optimizer="adam", loss="hinge")         # targets must be -1 / +1
```

3.5 Classification losses at a glance

Loss	Classes	Labels	Output layer	Keras name
Binary CE	2	0 / 1	1 neuron, sigmoid	binary_crossentropy
Categorical CE	3+	one-hot	k neurons, softmax	categorical_crossentropy
Sparse Categorical CE	3+ (many)	integers	k neurons, softmax	sparse_categorical_crossentropy
Hinge	2	−1 / +1	1 neuron, linear	hinge

KEY IDEA Summary of Part 3

• Classification outputs probabilities; loss = −log(probability of the true class). • **Binary CE**: 1 sigmoid neuron. • **Categorical CE**: k softmax neurons + one-hot labels. • **Sparse CE**: same loss, integer labels, faster. • **Hinge**: margins, labels ±1.

PART 4 • LEVEL 4 · ADVANCED

Advanced Losses: KL, GAN, Focal and Triplet

The video's map mentioned these four task-specific losses but did not explain them. Here each one gets the full treatment: intuition, formula, a worked number example, a picture and code.

4.1 KL Divergence (autoencoders and VAEs)

Autoencoders compress data into a small code and rebuild it. In **variational autoencoders (VAEs)** the code is not a single point but a **probability distribution**, and we want that distribution to look like a simple, well-behaved one (a standard bell curve). We need a number that measures **how different two probability distributions are**. That number is the **Kullback–Leibler (KL) divergence**.

ANALOGY Using the wrong map

P is the true map of a city; Q is the map your model believes. KL(P‖Q) measures how much *extra surprise* (extra bits of information) you suffer by navigating with Q when reality is P. If Q = P, you suffer none and KL = 0.

KL divergence from Q to P (sum over all outcomes x)

$$D_{KL}(P \,\|\, Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)}$$

Term by term: **P(x)** is the true probability of outcome x, **Q(x)** is the model's probability. The ratio P/Q says how wrong the model is about x. The log turns that into surprise, and we weight it by how often x really happens (P(x)).

EXAMPLE Worked example

True P = [0.5, 0.3, 0.2], model Q = [0.4, 0.4, 0.2].
KL(P‖Q) = 0.5·ln(0.5/0.4) + 0.3·ln(0.3/0.4) + 0.2·ln(0.2/0.2) = 0.5·0.2231 + 0.3·(-0.2877) + 0 = **0.0253**.
Swap the roles: KL(Q‖P) = 0.0258. **Not equal: KL is not symmetric**, so it is a "divergence", not a true distance.

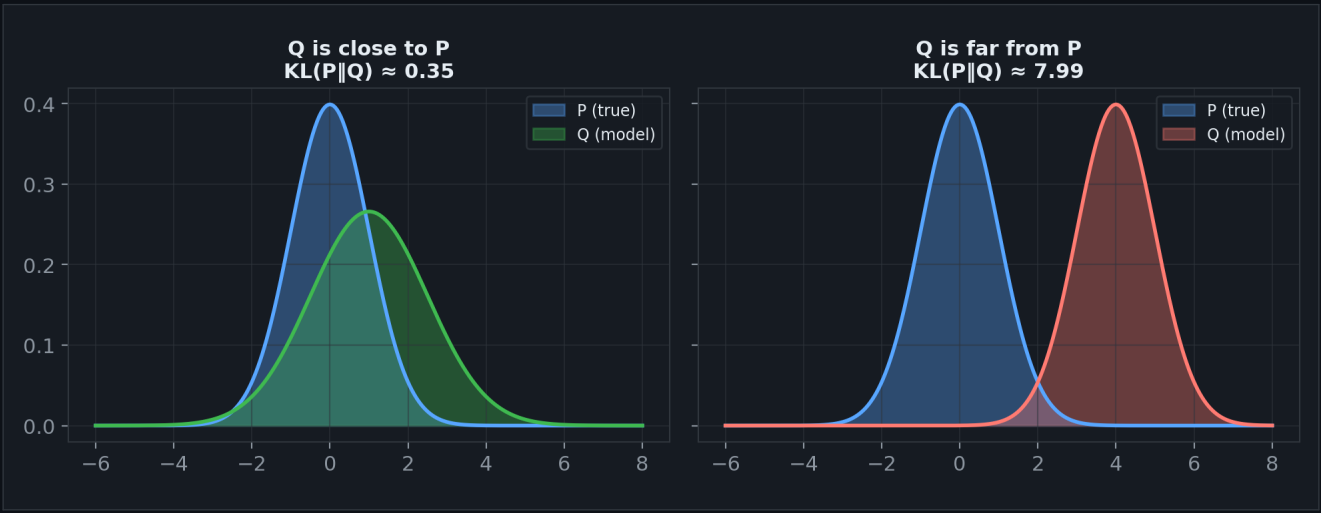


Figure 4.1 – The farther the model distribution Q (right) drifts from the truth P, the larger the KL divergence.

- KL ≥ 0 always, and KL = 0 only when the two distributions are identical.
- In a VAE the total loss is **reconstruction loss + KL term**: be accurate *and* keep the latent space tidy.
- Pro link: cross-entropy = entropy of P + KL(P‖Q). Since the entropy of the true labels is fixed, **minimising cross-entropy is the same as minimising KL divergence**. So Parts 3 and 4 are one family!

The VAE loss

$$L_{\text{VAE}} = \text{Reconstruction loss} + D_{KL}(q(z|x) \,\|\, \mathcal{N}(0,1))$$

Python (Keras / TensorFlow)

```python
kl = tf.keras.losses.KLDivergence()
print(kl([[.5,.3,.2]], [[.4,.4,.2]]).numpy())     # ~0.0253

# VAE: closed-form KL between N(mean, var) and N(0, 1)
kl_loss = -0.5 * tf.reduce_mean(1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var))
total_loss = reconstruction_loss + kl_loss
```

4.2 GAN losses: Discriminator loss and Minimax loss

A **Generative Adversarial Network** contains two networks that play a game.

ANALOGY Counterfeiter vs police

The **Generator** (G) is a counterfeiter who prints fake banknotes from random noise. The **Discriminator** (D) is a police officer who must tell real notes from fakes. Every time the police get better, the counterfeiter must improve, and vice versa. At the end the fakes are so good that D can only guess (50/50).

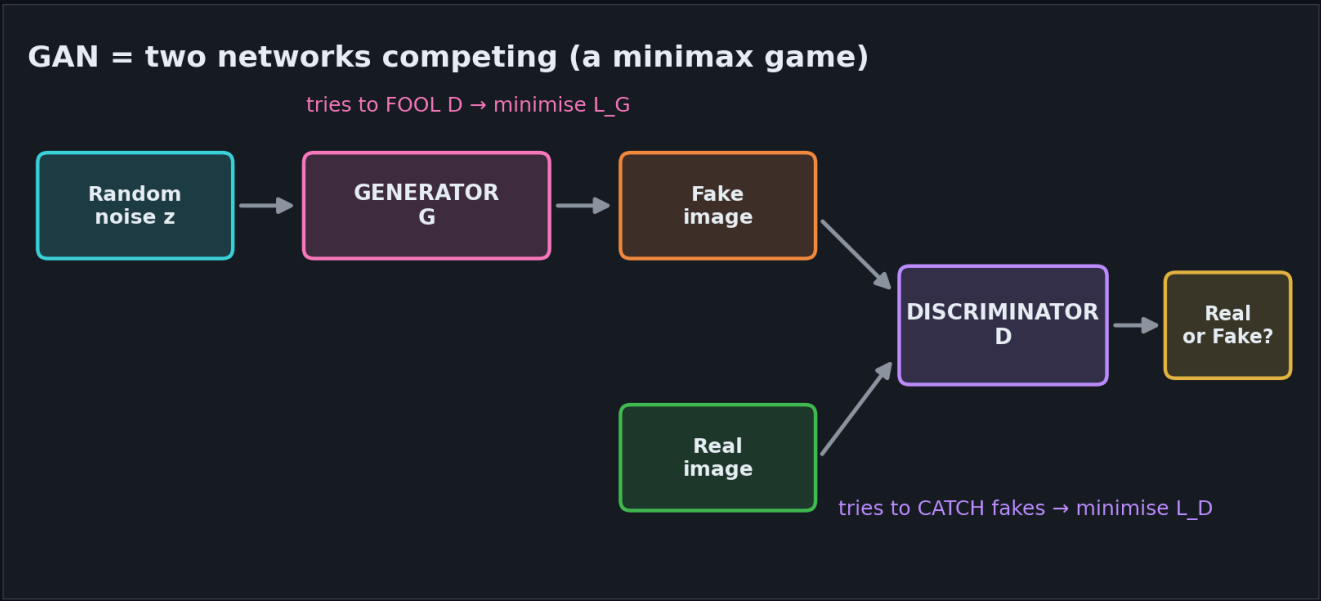


Figure 4.2 – The two networks of a GAN and who is trying to do what.

Each network has its own loss, and both use the binary cross-entropy you already know! D(x) is the probability, in [0, 1], that x is real.

Discriminator loss: say “real” (D→1) for real data x and “fake” (D→0) for generated G(z)

$$L_D = -[\log D(x) + \log(1 - D(G(z)))]$$

Generator loss (non-saturating form): make D say “real” for fakes

$$L_G = -\log D(G(z))$$

Put together, the original two-player game is the **Minimax GAN loss**: D tries to **maximise** the expression, G tries to **minimise** it.

Minimax GAN objective

$$\min_G \max_D \; \mathrm{E}_x[\log D(x)] + \mathrm{E}_z[\log(1 - D(G(z)))]$$

EXAMPLE Worked example

D says D(x) = 0.9 for a real image and D(G(z)) = 0.2 for a fake: $L_D = -[\ln 0.9 + \ln(1 - 0.2)] = -[-0.105 + -0.223] =$ **0.329** (D is doing well).

Early in training the generator is awful: D(G(z)) = 0.05. Non-saturating G loss = $-\ln 0.05 =$ **3.00** (large, strong learning signal). The original minimax form $\ln(1 - 0.05) = -0.051$ is almost flat, so G would barely learn. That is why practitioners use the non-saturating version.

Python (Keras / TensorFlow)

```python
bce = tf.keras.losses.BinaryCrossentropy()

def discriminator_loss(real_out, fake_out):
    return bce(tf.ones_like(real_out), real_out) + bce(tf.zeros_like(fake_out), fake_out)

def generator_loss(fake_out):
    return bce(tf.ones_like(fake_out), fake_out)     # pretend fakes are real
```

PRO LEVEL Why GANs are hard to train

The two losses fight each other, so neither goes steadily to zero. A falling D loss can mean G is failing. You judge a GAN by looking at generated samples (and metrics like FID), not by the loss curve alone. Variants such as Wasserstein GAN replace BCE with a smoother distance.

4.3 Focal Loss (object detection, imbalanced data)

In object detection, a picture has a few pixels/boxes with objects and **thousands of easy background examples**. If you use plain cross-entropy, those thousands of easy negatives, each with a small loss, add up and drown out the few hard, important examples.

ANALOGY A teacher's attention

A good teacher spends time on students who are struggling, not on the ones who already score 100%. Focal loss makes the network do the same: **ignore the easy cases, focus on the hard ones.**

Focal loss (p_t = probability given to the TRUE class)

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

It is cross-entropy, $-\log(p_t)$, multiplied by a **modulating factor $(1 - p_t)^{\gamma}$**. When the model is already confident and right (p_t close to 1), the factor is almost 0 and the example is silenced. When the model is wrong (p_t small), the factor is near 1 and the example keeps its full loss. γ (gamma, the "focusing parameter", typically 2) controls how aggressive the silencing is, and α_t is an optional class-balance weight.

p_t	Plain CE	Focal $\gamma = 2$	How much reduced
0.95	0.0513	0.0001	× 0.0025
0.9	0.1054	0.0011	× 0.0100
0.6	0.5108	0.0817	× 0.1600
0.3	1.2040	0.5899	× 0.4900
0.1	2.3026	1.8651	× 0.8100

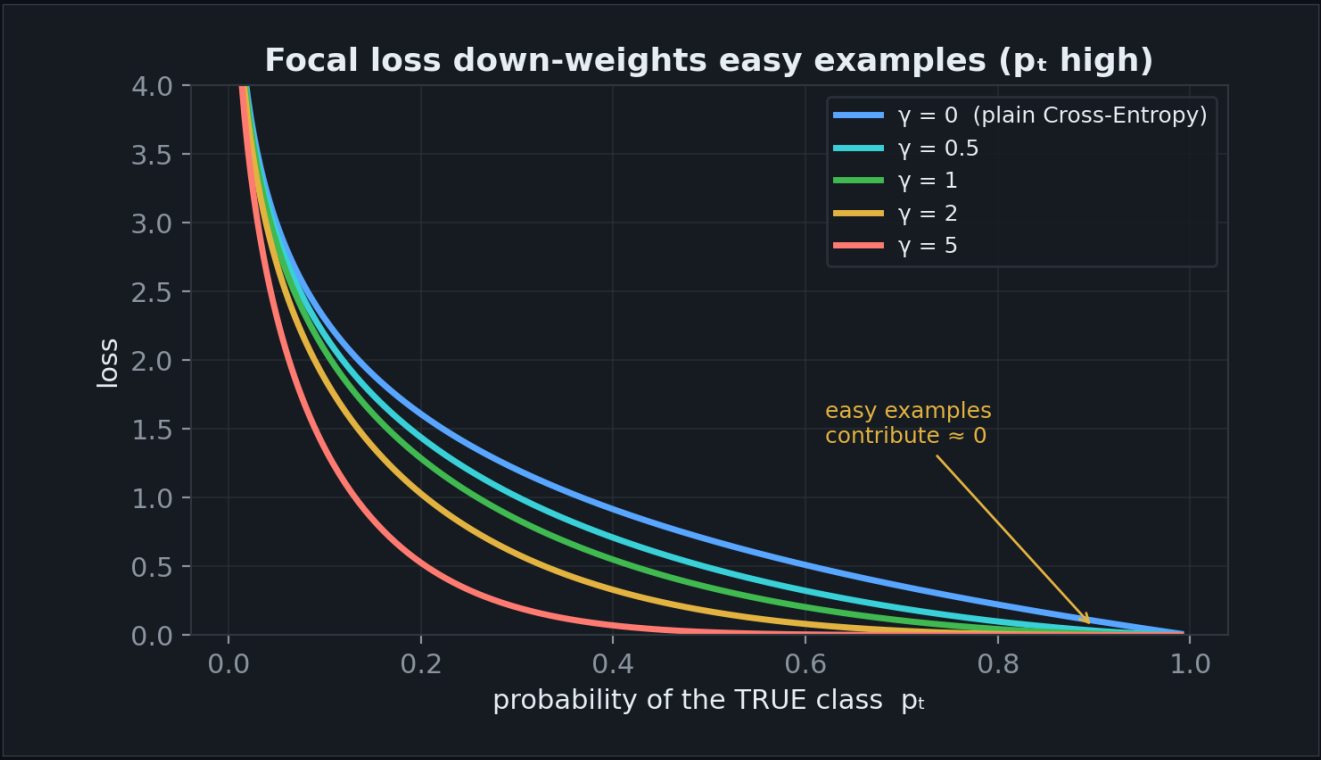


Figure 4.3 – Larger γ pushes the loss of well-classified examples (right side) towards zero.

Python (Keras / TensorFlow)

```python
def focal_loss(gamma=2.0, alpha=0.25):
    def loss(y_true, y_pred):
        y_pred = tf.clip_by_value(y_pred, 1e-7, 1 - 1e-7)
        pt = tf.where(tf.equal(y_true, 1), y_pred, 1 - y_pred)     # prob of the true class
        at = tf.where(tf.equal(y_true, 1), alpha, 1 - alpha)       # class-balance weight
        return tf.reduce_mean(-at * tf.pow(1 - pt, gamma) * tf.math.log(pt))
    return loss

model.compile(optimizer="adam", loss=focal_loss(gamma=2.0, alpha=0.25))
```

4.4 Triplet Loss (embeddings)

An **embedding** turns an item (a face, a word, a product) into a vector of numbers so that **similar items land close together**. There is no “class” to predict, so cross-entropy does not fit. Triplet loss trains the embedding by looking at **three** items at a time.

ANALOGY **Seating plan at a party**

Pick a guest (the **anchor**). Seat their friend (**positive**) close and a stranger (**negative**) far away, by at least a margin m.

Triplet loss (f = embedding network, a = anchor, p = positive, n = negative, m = margin)

$$L = \max\left(0,\ \|f(a) - f(p)\|^2 - \|f(a) - f(n)\|^2 + m\right)$$



Figure 4.4 – Training pulls the positive towards the anchor and pushes the negative away until the margin is satisfied.

EXAMPLE Worked example (m = 0.2)

Before training: $\|f(a)-f(p)\|^2 = 0.9$, $\|f(a)-f(n)\|^2 = 0.7$ → loss = max(0, 0.9 − 0.7 + 0.2) = **0.4** (the stranger is closer than the friend!).

After training: $\|f(a)-f(p)\|^2 = 0.2$, $\|f(a)-f(n)\|^2 = 1.5$ → loss = max(0, 0.2 − 1.5 + 0.2) = **0**. Nothing left to fix for this triplet.

Python (Keras / TensorFlow)

```python
def triplet_loss(anchor, positive, negative, margin=0.2):
    d_pos = tf.reduce_sum(tf.square(anchor - positive), axis=1)
    d_neg = tf.reduce_sum(tf.square(anchor - negative), axis=1)
    return tf.reduce_mean(tf.maximum(d_pos - d_neg + margin, 0.0))
```

PRO LEVEL Hard triplet mining

Most random triplets are already satisfied (loss 0) and teach nothing. Real systems (FaceNet) pick **hard** or **semi-hard** triplets, where the negative is close to the anchor, to make training efficient.

KEY IDEA Summary of Part 4

• **KL**: distance between distributions (VAE). • **GAN**: two losses in a game, built from BCE. • **Focal**: cross-entropy that ignores easy examples. • **Triplet**: pull friends close, push strangers away.

PART 5 • LEVEL 5 · PRO

Pro Level: Gradients, Stability and Choosing Wisely

Now we look under the hood: what the loss does to the gradients, how to avoid numerical disasters, how to debug, how to write your own loss, and how to choose with confidence.

5.1 Why cross-entropy beats MSE for classification

Could you use MSE for classification? Technically yes, but training is much slower. The reason lies in the **gradient** with respect to the raw score z (the value just before the sigmoid/softmax).

Sigmoid + BCE, and Softmax + CCE: the gradient is simply (prediction − truth)

$$\frac{\partial L}{\partial z_j} = \hat{y}_j - y_j$$

This elegant result is why the pairing **sigmoid with BCE** and **softmax with CCE** is the standard. The log in the loss cancels the exponential in the activation, so the gradient is proportional to the error. With MSE, an extra factor $\hat{y}(1-\hat{y})$ appears, and it is nearly zero when the neuron is saturated.

Case: y = 1, model says ŷ = 0.01 (confidently wrong)	Gradient size wrt z
BCE: ŷ − y	0.990 (strong push)
MSE with sigmoid: (ŷ − y)·ŷ(1 − ŷ) (factor 2 dropped)	0.0098 (about 101× weaker: a stalled model)

5.2 Numerical stability

- **log(0) is −∞.** If a prediction reaches exactly 0 or 1, the loss becomes infinite and training collapses into NaN. Libraries clip predictions to [ε, 1 − ε] with ε ≈ 1e-7.
- **Use logits when possible.** Computing sigmoid/softmax and then log in separate steps loses precision. Let the loss do both in one stable step with `from_logits=True`.
- **Softmax overflow.** e^{1000} overflows, so implementations subtract the maximum score first (the result is unchanged).

Python (Keras / TensorFlow)

```
# stable version: the last layer has NO activation (it outputs logits)
model = Sequential([Dense(64, activation="relu"), Dense(10)])
model.compile(optimizer="adam",
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=["accuracy"])
```

5.3 Loss is not the same as a metric

The **loss** is what the optimiser minimises, so it must be differentiable. A **metric** (accuracy, F1, RMSE in LPA) is what *humans* care about. Accuracy is flat between predictions, so it gives no gradient. You train on cross-entropy and report accuracy. A falling loss with a flat accuracy is normal in the early epochs, and a loss on the validation set that rises while the training loss falls signals **over-fitting**.

5.4 Regularisation lives inside the loss

To fight over-fitting you can add a penalty on large weights directly to the cost: **total loss = data loss + λ · penalty(weights)**. L2 regularisation (weight decay) uses the sum of squared weights, L1 uses the sum of absolute weights, and λ sets the strength. Keras reports this extra term automatically when you add `kernel_regularizer` to a layer. Note the nice symmetry: L2/L1 here are the same ideas as the L2 (MSE) and L1 (MAE) losses, applied to the weights instead of the errors.

5.5 Class imbalance

- Use **class weights**: `model.fit(..., class_weight={0: 1.0, 1: 10.0})` multiplies the loss of rare-class examples.
- Use **Focal loss** (Part 4.3) to down-weight the easy majority.
- Judge with precision / recall / F1 / AUC, not accuracy: a model that says “no fraud” every time is 99% accurate on a 1%-fraud dataset.

5.6 Sanity checks that save hours

1. **Check the starting loss.** With random weights and k balanced classes, cross-entropy should be about ln(k). For 10 classes that is ln 10 = **2.303**. A very different number means a bug (wrong activation, wrong labels).
2. **Overfit one small batch.** The loss should plunge to almost 0. If not, something is broken in the model, data or loss.
3. **Loss = NaN?** Learning rate too high, log(0), division by zero, or unnormalised inputs/targets.
4. **Scale your targets.** If y is in the millions, MSE explodes. Standardise y (or log-transform it) and invert after predicting.
5. **Mean or sum?** Keras averages over the batch by default. If you write a custom loss, be consistent, otherwise your effective learning rate silently changes.

5.7 Writing your own loss in Keras

A loss is just a function `f(y_true, y_pred)` that returns a number (or one per example) built from differentiable TensorFlow operations. Example: an asymmetric loss that punishes **under-predicting** a package twice as much as over-predicting it.

Custom loss functions

```
def asymmetric_mse(y_true, y_pred):
    err = y_true - y_pred                       # >0 means we under-predicted
    weight = tf.where(err > 0, 2.0, 1.0)        # under-prediction costs double
    return tf.reduce_mean(weight * tf.square(err))

model.compile(optimizer="adam", loss=asymmetric_mse)

# a configurable loss (a "factory") returning the function
def weighted_huber(delta=1.0, w=1.0):
    h = tf.keras.losses.Huber(delta=delta)
    return lambda y_true, y_pred: w * h(y_true, y_pred)
```

5.8 Choosing the right loss: the decision map

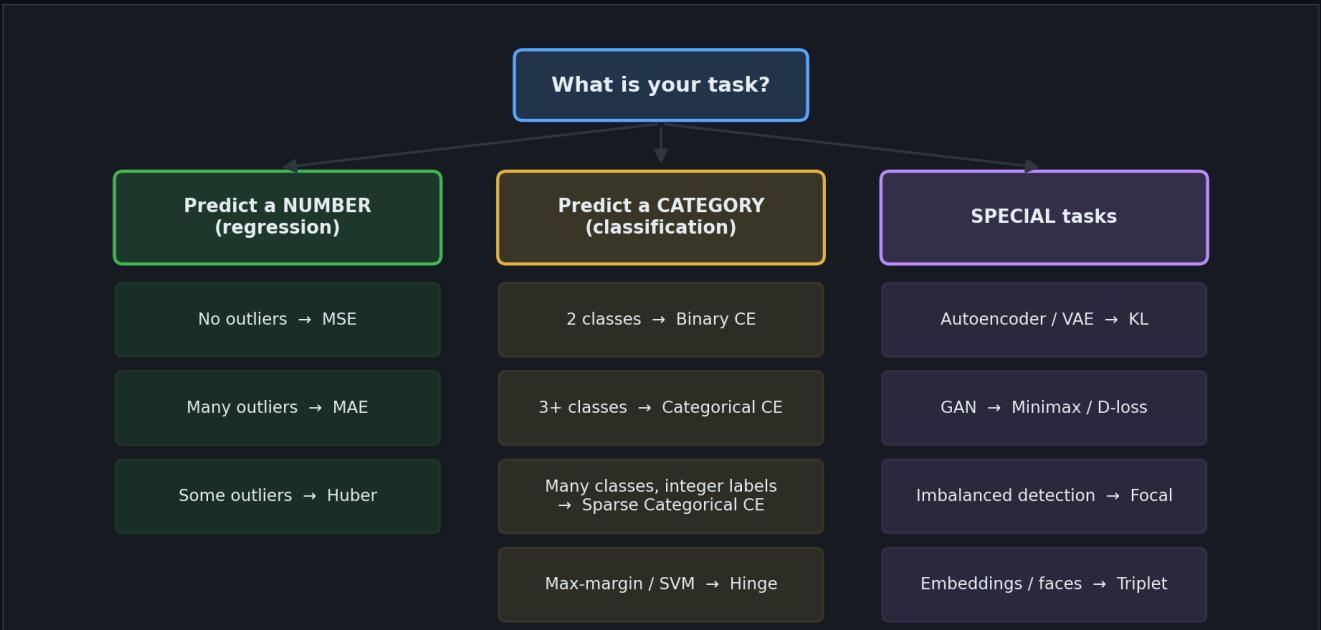


Figure 5.1 – Start from your task and follow the branch.

Output activation and loss must be a matched pair

Task	Output layer	Loss	Labels
Regression	1 neuron, linear	mse / mae / Huber	real numbers
Binary classification	1 neuron, sigmoid	binary_crossentropy	0 / 1
Multi-class (single label)	k neurons, softmax	categorical_crossentropy	one-hot
Multi-class (single label)	k neurons, softmax	sparse_categorical_crossentropy	integers
Multi-label (several tags at once)	k neurons, sigmoid	binary_crossentropy	k-long 0/1 vector
SVM-style margin	1 neuron, linear	hinge	−1 / +1
Probability distributions (VAE)	distribution parameters	KL divergence	–

WATCH OUT Most common beginner bugs

• Softmax + binary_crossentropy for a multi-class task. • Sigmoid output with 3+ mutually exclusive classes. • one-hot labels with the *sparse* loss (or integers with the non-sparse one). • A ReLU/sigmoid output layer in regression.

5.9 The master cheat sheet

Loss	Task	Per-example formula	Output	Outliers	Keras
MSE (L2)	regression	$(y - \hat{y})^2$	linear	sensitive	mse
MAE (L1)	regression	$\lvert y - \hat{y} \rvert$	linear	robust	mae
Huber	regression	$\frac{1}{2}e^2$ or $\delta\lvert e \rvert - \frac{1}{2}\delta^2$	linear	robust	Huber
Binary CE	2 classes	$-y \log \hat{y} - (1-y) \log(1-\hat{y})$	sigmoid	–	binary_crossentropy
Categorical CE	k classes	$-\Sigma\, y_j \log \hat{y}_j$	softmax	–	categorical_crossentropy
Sparse Cat. CE	k classes (ints)	$-\log \hat{y}_c$	softmax	–	sparse_categorical_crossentropy
Hinge	SVM / margins	$\max(0, 1 - y\, f)$	linear	–	hinge
KL divergence	distributions	$\Sigma\, P \log(P/Q)$	softmax / params	–	KLDivergence
GAN (D, G)	generation	BCE-based	sigmoid (D)	–	custom
Focal	imbalanced detection	$-\alpha(1-p_t)^{\gamma} \log p_t$	sigmoid / softmax	–	custom
Triplet	embeddings	$\max(0, d^+ - d^- + m)$	embedding vector	–	custom

5.10 Interview questions with model answers

INTERVIEW Q1. What is the difference between a loss function and a cost function?
Loss is for one training example; cost is the average of the losses over the whole training set (or batch). We minimise the cost.

INTERVIEW Q2. Why do we square the error in MSE?
So that positive and negative errors do not cancel, and so that big errors are penalised more. It also makes the function smooth and differentiable, with a single minimum for linear models.

INTERVIEW Q3. When would you prefer MAE (or Huber) over MSE?
When the data has outliers. MAE grows linearly, so outliers do not dominate; Huber is smooth near zero like MSE and linear for large errors, with δ set by you.

INTERVIEW Q4. Why can't we use MSE for classification?
It can be used, but with sigmoid/softmax outputs its gradient is multiplied by $\hat{y}(1 - \hat{y})$, which vanishes when the model is confidently wrong, so learning stalls. Cross-entropy gives a gradient of $(\hat{y} - y)$.

INTERVIEW Q5. What activation goes with each loss?
MSE/MAE/Huber: linear. Binary CE: sigmoid on one neuron. Categorical and Sparse CE: softmax on k neurons.

INTERVIEW Q6. Categorical vs Sparse Categorical cross-entropy?
Same loss and architecture. Categorical needs one-hot labels, Sparse uses integer labels and is faster and lighter when there are many classes.

INTERVIEW Q7. What does KL divergence measure and is it symmetric?
How different distribution Q is from P: Σ P log(P/Q). It is ≥ 0 and zero only when P = Q, but it is not symmetric.

INTERVIEW Q8. What is the starting loss of a 10-class classifier?
About ln(10) $\approx$ 2.30 if it begins by guessing uniformly.

5.11 Practice problems (answers included)

#	Problem	Answer
1	y = [3, 5, 2], $\hat{y}$ = [2, 7, 2]. Find MSE and MAE.	MSE = (1 + 4 + 0)/3 = **1.667**; MAE = (1 + 2 + 0)/3 = **1.000**
2	BCE (ln): (a) y = 1, $\hat{y}$ = 0.8 (b) y = 0, $\hat{y}$ = 0.3	(a) $-\ln 0.8$ = **0.223** (b) $-\ln 0.7$ = **0.357**
3	Three classes, true = class 2, $\hat{y}$ = [0.1, 0.7, 0.2]. CCE?	$-\ln 0.7$ = **0.357**
4	Huber δ = 2: error 5 and error 1?	error 5: $2 \cdot 5 - \frac{1}{2} \cdot 4$ = **8**; error 1: $\frac{1}{2} \cdot 1$ = **0.5**
5	Softmax of z = [1, 2, 3]?	**[0.090, 0.245, 0.665]**
6	Hinge, y = -1, f(x) = 0.4?	max(0, 1 $-$ (-1)(0.4)) = **1.4**
7	Focal (γ = 2), p_t = 0.9: factor vs plain CE?	$(1 - 0.9)^2$ = **0.01**, so the loss is 100× smaller (0.1054 → 0.00105)
8	Why is MSE a bad idea if one target is 1,000,000 while the rest are about 5?	The squared error of the outlier dwarfs everything and dominates the gradient: use MAE/Huber or scale/log-transform the targets.

5.12 Did we cover everything in the video?

Every topic from the lecture script is included, reorganised and expanded:

Topic in the video	Where in this tutorial
Definition of a loss function, depends on parameters, linear-regression example	1.1
Why it is important (Drucker quote, “eye of the algorithm”)	1.2
Role in deep learning: forward propagation → loss → gradient descent, epochs	1.3
Types of losses chart (regression, classification, autoencoders, GANs, detection, embeddings), custom losses in Keras	1.5, Part 4, 5.7
Loss vs Cost function (+ error function, interview tip)	1.4
MSE: formula, why square, quadratic punishment, advantages, disadvantages, outliers, linear last neuron, Keras	2.1
MAE: formula, cost, advantages, non-differentiability / sub-gradients	2.2
Huber: idea (25% outliers), δ hyperparameter, MSE-like vs MAE-like	2.3
Binary Cross-Entropy: formula, sigmoid, manual calculation, log loss, derivation pointer, pros and cons	3.1
Categorical Cross-Entropy: softmax, k neurons, one-hot, worked examples, MNIST, cost formula	3.2
Sparse Categorical Cross-Entropy: integer labels, same loss, faster	3.3
Summary: which loss to use when	2.4, 3.5, 5.8, 5.9
Hinge (SVM), KL, GAN, Focal, Triplet (named but not explained in the video)	3.4, Part 4

KEY IDEA **Final takeaway**

A loss function is the **compass** of learning. Regression: **MSE** (clean data), **MAE** (outliers), **Huber** (mix). Classification: **Binary CE** (2 classes, sigmoid), **Categorical / Sparse CE** (many classes, softmax). Specialised problems: **KL, GAN, Focal, Triplet**. Match the loss with the right output activation and label format, and you are ready for the next topic: **Backpropagation**.